

**SIMATS ENGINEERING
ASSESSMENT TOOL 3**

COURSE NAME: Fundamentals of Data Science

COURSE CODE: DSA04

COURSE OUTCOME COVERED

***CO1:** Analyse the role of data science, facets of data, and the big data ecosystem for informed decision-making. (BL2)*

QUESTIONS: 05

Total Marks:100

Annexure A

Exploratory Data Analysis - Based Practical Assessment

Code of Conduct:

*I K. SAI DATTA (192472016) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

Exploratory Data Analysis-Based Practical Assessment

1. Education Analytics Dataset

Question 1

A college wants to analyze student performance based on attendance, study hours, internal marks, and final result. Perform EDA to identify the factors affecting student results.

Student_ID	Attendance	Study_Hours	Internal_Marks	Result
S01	92	5	45	Pass
S02	55	2	24	Fail
S03	78	4	38	Pass
S04	60	3	28	Fail
S05	88	5	42	Pass
S06	47	1	20	Fail
S07	95	6	48	Pass
S08	72	3	35	Pass
S09	50	2	22	Fail
S10	83	4	40	Pass
S11	66	3	30	Fail
S12	90	5	44	Pass
S13	58	2	25	Fail
S14	76	4	36	Pass
S15	62	3	29	Fail
S16	85	5	41	Pass
S17	45	1	18	Fail
S18	80	4	39	Pass
S19	68	3	32	Pass
S20	53	2	23	Fail

Code:

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
# Create the dataset
```

```
data = {  
    "Student_ID": ["S01","S02","S03","S04","S05","S06","S07","S08","S09","S10",  
                   "S11","S12","S13","S14","S15","S16","S17","S18","S19","S20"],  
    "Attendance": [92,55,78,60,88,47,95,72,50,83,66,90,58,76,62,85,45,80,68,53],  
    "Study_Hours": [5,2,4,3,5,1,6,3,2,4,3,5,2,4,3,5,1,4,3,2],  
    "Internal_Marks": [45,24,38,28,42,20,48,35,22,40,30,44,25,36,29,41,18,39,32,23],  
    "Result": ["Pass","Fail","Pass","Fail","Pass","Fail","Pass","Pass","Fail","Pass",  
               "Fail","Pass","Fail","Pass","Fail","Pass","Fail","Pass","Pass","Fail"]  
}
```

```
df = pd.DataFrame(data)
```

```
# Display dataset
```

```

print("Student Dataset")
print(df)

# Basic information
print("\nDataset Information")
print(df.info())

# Statistical summary
print("\nStatistical Summary")
print(df.describe())

# Count Pass and Fail
print("\nResult Count")
print(df["Result"].value_counts())

# Average values based on Result
print("\nAverage Values")
print(df.groupby("Result")[["Attendance", "Study_Hours", "Internal_Marks"]].mean())

# Plot Attendance
plt.figure(figsize=(5,4))
plt.bar(df["Student_ID"], df["Attendance"])
plt.title("Attendance of Students")
plt.xlabel("Student ID")
plt.ylabel("Attendance")
plt.xticks(rotation=90)
plt.show()

# Plot Study Hours
plt.figure(figsize=(5,4))
plt.bar(df["Student_ID"], df["Study_Hours"])
plt.title("Study Hours")
plt.xlabel("Student ID")
plt.ylabel("Hours")
plt.xticks(rotation=90)
plt.show()

# Plot Internal Marks
plt.figure(figsize=(5,4))
plt.bar(df["Student_ID"], df["Internal_Marks"])
plt.title("Internal Marks")
plt.xlabel("Student ID")
plt.ylabel("Marks")
plt.xticks(rotation=90)
plt.show()

# Pie Chart for Result
plt.figure(figsize=(5,5))
df["Result"].value_counts().plot(kind="pie", autopct="%1.1f%%")
plt.title("Pass vs Fail")

```

```
plt.ylabel("")
plt.show()
```

```
# Conclusion
```

```
print("\nConclusion:")
```

```
print("Students with higher attendance, more study hours, and higher internal marks mostly passed.")
```

```
print("Students with low attendance, fewer study hours, and low internal marks mostly failed.")
```

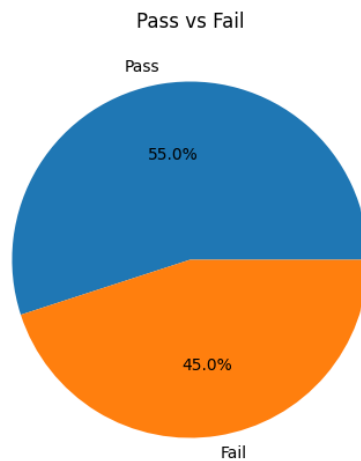
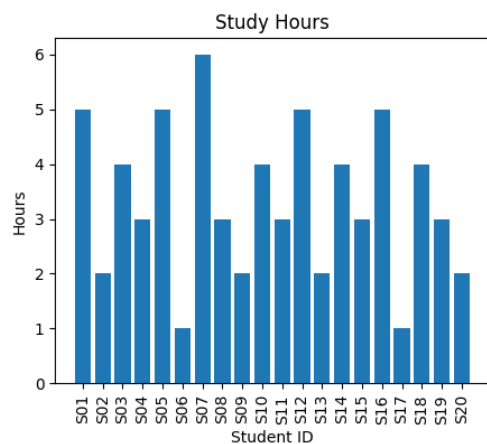
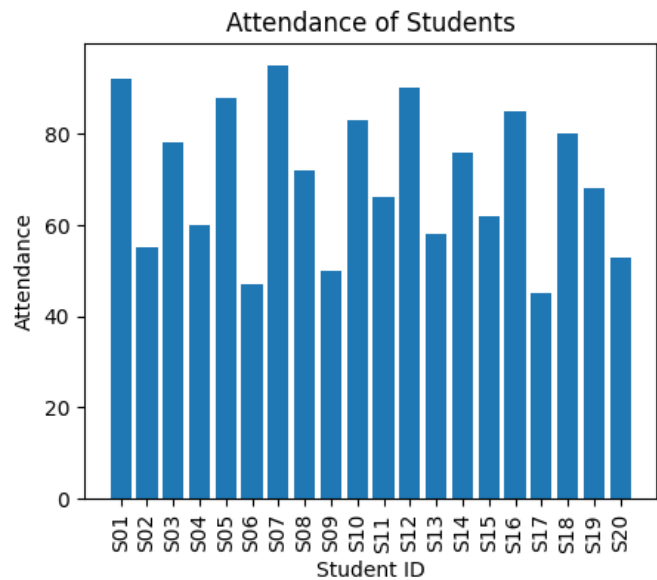
```
Output:
```

```
Saving student.csv.xlsx to student.csv (1).xlsx
First 5 Records
Student_ID  Attendance  Study_Hours  Internal_Marks  Result
0          S001         92             5              45      Pass
1          S002         55             2              24      Fail
2          S003         78             4              38      Pass
3          S004         60             3              28      Fail
4          S005         88             5              42      Pass

Shape: (20, 5)

Data Types:
Student_ID      object
Attendance      int64
Study_Hours     int64
Internal_Marks  int64
Result          object
dtype: object

Missing Values:
Student_ID      0
Attendance      0
Study_Hours     0
Internal_Marks  0
Result          0
...
25%    57.250000    2.000000    24.750000
50%    70.000000    3.000000    33.500000
75%    83.500000    4.250000    40.250000
max     95.000000    6.000000    48.000000
```



2. Healthcare Analytics Dataset

Question 2

A hospital wants to analyze patient health risk based on age, blood sugar, blood pressure, BMI, and disease status. Perform EDA to find important health patterns.

Patient_ID	Age	Sugar_Level	BP	BMI	Disease_Status
P01	45	145	130	28	Yes
P02	32	98	118	23	No
P03	55	180	145	31	Yes
P04	28	90	110	22	No
P05	60	200	150	34	Yes
P06	40	120	125	26	No
P07	52	165	140	30	Yes
P08	35	105	120	24	No
P09	48	155	135	29	Yes
P10	30	95	115	23	No
P11	62	210	155	35	Yes
P12	38	115	122	25	No
P13	50	170	142	30	Yes
P14	27	88	108	21	No
P15	58	190	148	33	Yes
P16	42	130	128	27	No
P17	46	150	132	28	Yes
P18	34	100	116	24	No
P19	53	175	144	31	Yes
P20	29	92	112	22	No

Code:

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
# Create the dataset
```

```
data = {  
    "Patient_ID": ["P01","P02","P03","P04","P05","P06","P07","P08","P09","P10",  
                   "P11","P12","P13","P14","P15","P16","P17","P18","P19","P20"],  
    "Age": [45,32,55,28,60,40,52,35,48,30,62,38,50,27,58,42,46,34,53,29],  
    "Sugar_Level":  
[145,98,180,90,200,120,165,105,155,95,210,115,170,88,190,130,150,100,175,92],  
    "BP":  
[130,118,145,110,150,125,140,120,135,115,155,122,142,108,148,128,132,116,144,112],  
    "BMI": [28,23,31,22,34,26,30,24,29,23,35,25,30,21,33,27,28,24,31,22],  
    "Disease_Status": ["Yes","No","Yes","No","Yes","No","Yes","No","Yes","No",  
                       "Yes","No","Yes","No","Yes","No","Yes","No","Yes","No"]  
}
```

```
df = pd.DataFrame(data)
```

```
# Display dataset
```

```
print("Healthcare Dataset")
```

```
print(df)
```

```

# Dataset Information
print("\nDataset Information")
print(df.info())

# Statistical Summary
print("\nStatistical Summary")
print(df.describe())

# Disease Status Count
print("\nDisease Status Count")
print(df["Disease_Status"].value_counts())

# Average values based on Disease Status
print("\nAverage Values")
print(df.groupby("Disease_Status")[["Age", "Sugar_Level", "BP", "BMI"]].mean())

# Bar Chart - Sugar Level
plt.figure(figsize=(6,4))
plt.bar(df["Patient_ID"], df["Sugar_Level"])
plt.title("Patient Sugar Level")
plt.xlabel("Patient ID")
plt.ylabel("Sugar Level")
plt.xticks(rotation=90)
plt.show()

# Bar Chart - Blood Pressure
plt.figure(figsize=(6,4))
plt.bar(df["Patient_ID"], df["BP"])
plt.title("Blood Pressure")
plt.xlabel("Patient ID")
plt.ylabel("BP")
plt.xticks(rotation=90)
plt.show()

# Bar Chart - BMI
plt.figure(figsize=(6,4))
plt.bar(df["Patient_ID"], df["BMI"])
plt.title("BMI of Patients")
plt.xlabel("Patient ID")
plt.ylabel("BMI")
plt.xticks(rotation=90)
plt.show()

# Pie Chart - Disease Status
plt.figure(figsize=(5,5))
df["Disease_Status"].value_counts().plot(kind="pie", autopct="%1.1f%%")
plt.title("Disease Status")
plt.ylabel("")
plt.show()

```

```
print("Patients with higher sugar levels, blood pressure, BMI, and age are more likely to have the disease.")
print("Patients with normal health parameters mostly do not have the disease.")
```

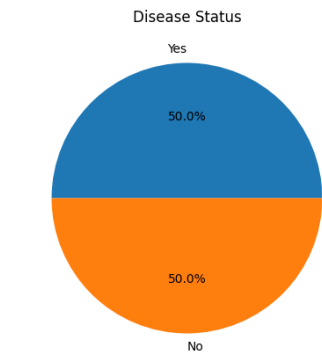
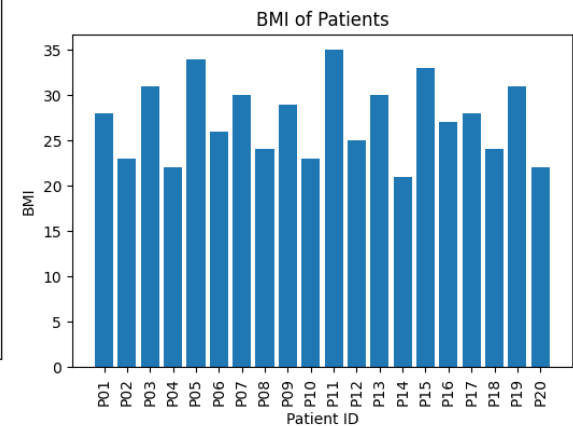
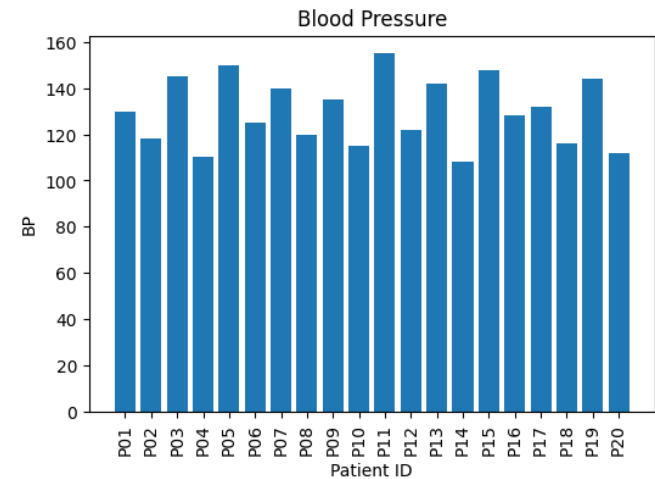
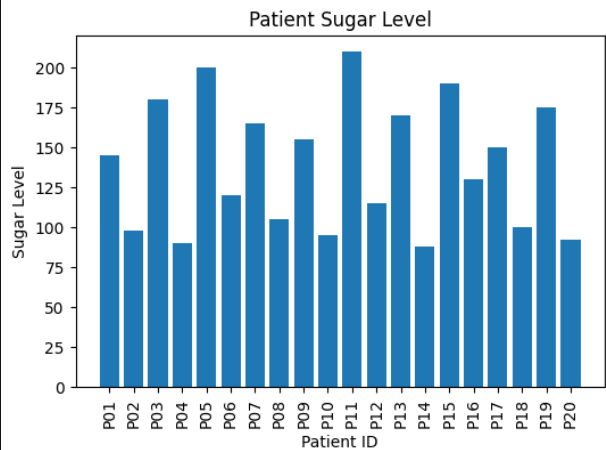
Output:

```
Saving healthcare.csv.xlsx to healthcare.csv (1).xlsx
First 5 Records
Patient_ID  Age  Sugar_Level  BP  BMI  Disease_Status
0      P01   45        145  130  28           Yes
1      P02   32         98  118  23           No
2      P03   55        180  145  31           Yes
3      P04   28         90  110  22           No
4      P05   60        200  150  34           Yes

Shape: (20, 6)

Data Types:
Patient_ID      object
Age             int64
Sugar_Level     int64
BP              int64
BMI             int64
Disease_Status  object
dtype: object

Missing Values:
Patient_ID      0
Age             0
Sugar_Level     0
BP              0
...
Sugar_Level     40.334490
BP              14.523574
BMI             4.256265
dtype: float64
```



3. Retail Sales Dataset

Question 3

A supermarket wants to analyze product sales based on category, price, quantity sold, discount, and revenue. Perform EDA to identify high-selling products and revenue patterns.

Product_ID	Category	Price	Quantity_Sold	Discount	Revenue
PR01	Grocery	50	20	5	1000
PR02	Snacks	30	35	3	1050
PR03	Dairy	45	18	2	810
PR04	Fruits	60	25	5	1500
PR05	Vegetables	40	30	4	1200
PR06	Grocery	80	15	6	1200
PR07	Snacks	25	40	3	1000
PR08	Dairy	55	16	2	880
PR09	Fruits	70	22	5	1540
PR10	Vegetables	35	28	4	980
PR11	Grocery	90	12	6	1080
PR12	Snacks	20	45	2	900
PR13	Dairy	65	14	3	910
PR14	Fruits	75	20	5	1500
PR15	Vegetables	45	26	4	1170
PR16	Grocery	100	10	7	1000
PR17	Snacks	35	32	3	1120
PR18	Dairy	50	19	2	950
PR19	Fruits	80	18	6	1440
PR20	Vegetables	30	35	3	1050

Code:

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
# Create the dataset
```

```
data = {
```

```
    "Product_ID":
```

```
["PR01","PR02","PR03","PR04","PR05","PR06","PR07","PR08","PR09","PR10",
```

```
    "PR11","PR12","PR13","PR14","PR15","PR16","PR17","PR18","PR19","PR20"],
```

```
    "Category":
```

```
["Grocery","Snacks","Dairy","Fruits","Vegetables","Grocery","Snacks","Dairy","Fruits","Vegetables",
```

```
"Grocery","Snacks","Dairy","Fruits","Vegetables","Grocery","Snacks","Dairy","Fruits","Vegetables"],
```

```
    "Price": [50,30,45,60,40,80,25,55,70,35,90,20,65,75,45,100,35,50,80,30],
```

```
    "Quantity_Sold": [20,35,18,25,30,15,40,16,22,28,12,45,14,20,26,10,32,19,18,35],
```

```
    "Discount": [5,3,2,5,4,6,3,2,5,4,6,2,3,5,4,7,3,2,6,3],
```

```
    "Revenue":
```

```
[1000,1050,810,1500,1200,1200,1000,880,1540,980,1080,900,910,1500,1170,1000,1120,950,1440,1050]
```

```
}
```



```

df = pd.DataFrame(data)

# Display dataset
print("Retail Sales Dataset")
print(df)

# Dataset Information
print("\nDataset Information")
print(df.info())

# Statistical Summary
print("\nStatistical Summary")
print(df.describe())

# Average values by Category
print("\nAverage Values by Category")
print(df.groupby("Category")[["Price", "Quantity_Sold", "Discount", "Revenue"]].mean())

# Highest Revenue Product
print("\nHighest Revenue Product")
print(df.loc[df["Revenue"].idxmax()])

# Highest Quantity Sold Product
print("\nHighest Quantity Sold Product")
print(df.loc[df["Quantity_Sold"].idxmax()])

# Revenue Bar Chart
plt.figure(figsize=(7,4))
plt.bar(df["Product_ID"], df["Revenue"])
plt.title("Revenue by Product")
plt.xlabel("Product ID")
plt.ylabel("Revenue")
plt.xticks(rotation=90)
plt.show()

# Quantity Sold Bar Chart
plt.figure(figsize=(7,4))
plt.bar(df["Product_ID"], df["Quantity_Sold"])
plt.title("Quantity Sold")
plt.xlabel("Product ID")
plt.ylabel("Quantity")
plt.xticks(rotation=90)
plt.show()

# Revenue by Category
category_revenue = df.groupby("Category")["Revenue"].sum()

plt.figure(figsize=(6,4))
category_revenue.plot(kind="bar")
plt.title("Revenue by Category")

```

```
plt.xlabel("Category")
plt.ylabel("Total Revenue")
plt.show()
```

```
# Pie Chart - Category Distribution
```

```
plt.figure(figsize=(6,6))
df["Category"].value_counts().plot(kind="pie", autopct="%1.1f%%")
plt.title("Product Categories")
plt.ylabel("")
plt.show()
```

```
# Conclusion
```

```
print("\nConclusion:")
print("Fruits generate the highest revenue.")
print("Snacks have the highest quantity sold.")
print("Higher discounts do not always result in higher revenue.")
```

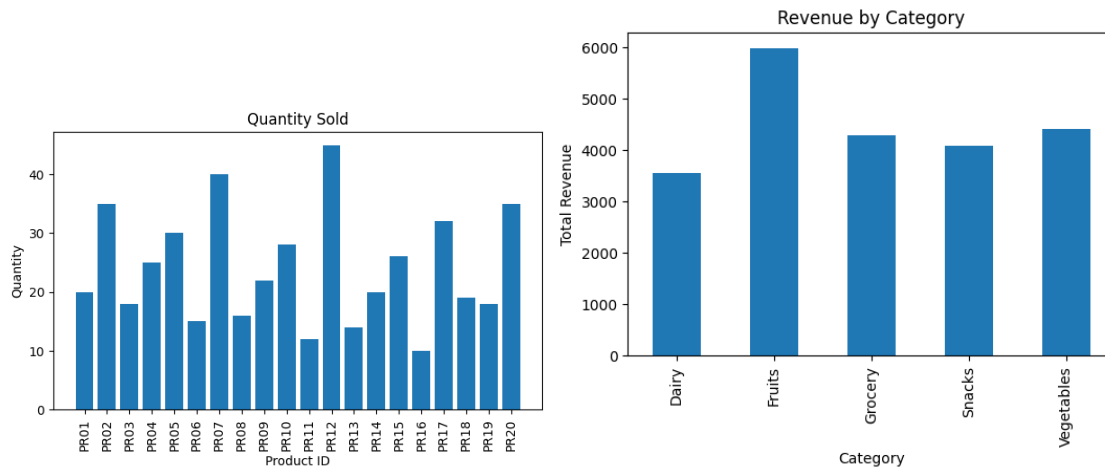
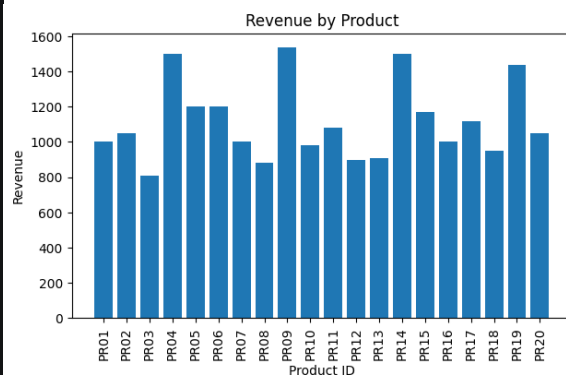
Output:

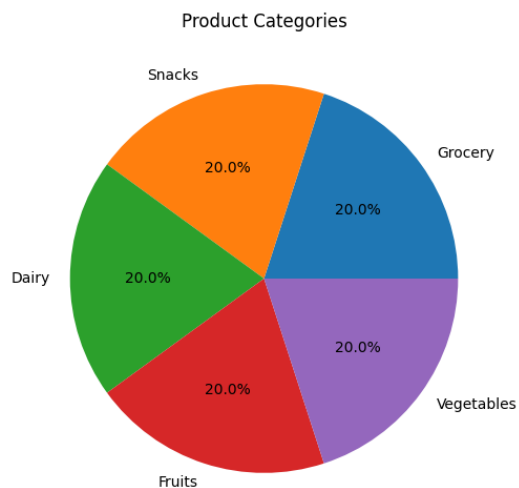
```
Saving retail.csv.xlsx to retail.csv.xlsx
First 5 Records
Product_ID  Category  Price  Quantity_Sold  Discount  Revenue
0          PR01  Grocery    50             20         5        1000
1          PR02   Snacks    30             35         3       1050
2          PR03   Dairy    45             18         2         810
3          PR04   Fruits    60             25         5       1500
4          PR05  Vegetables  40             30         4       1200

Shape: (20, 6)

Data Types:
Product_ID    object
Category       object
Price         int64
Quantity_Sold int64
Discount       int64
Revenue        int64
dtype: object

Missing Values:
Product_ID    0
Category       0
Price         0
Quantity_Sold 0
...
Quantity_Sold 9.673621
Discount      1.555973
Revenue       221.416493
dtype: float64
```





4. Banking Loan Approval Dataset

Question 4

A bank wants to analyze loan approval decisions based on income, credit score, loan amount, employment type, and approval status. Perform EDA to identify patterns related to loan approval.

Customer_ID	Income	Credit_Score	Loan_Amount	Employment_Type	Loan_Status
C01	45000	720	200000	Salaried	Approved
C02	25000	580	150000	Self-employed	Rejected
C03	60000	750	300000	Salaried	Approved
C04	22000	560	120000	Unemployed	Rejected
C05	50000	710	250000	Salaried	Approved
C06	30000	600	180000	Self-employed	Rejected
C07	70000	780	350000	Salaried	Approved
C08	28000	590	160000	Self-employed	Rejected
C09	55000	730	270000	Salaried	Approved
C10	24000	570	130000	Unemployed	Rejected
C11	65000	760	320000	Salaried	Approved
C12	35000	620	200000	Self-employed	Rejected
C13	48000	700	240000	Salaried	Approved
C14	26000	585	140000	Unemployed	Rejected
C15	75000	790	400000	Salaried	Approved
C16	32000	610	190000	Self-employed	Rejected
C17	52000	725	260000	Salaried	Approved
C18	27000	575	150000	Self-employed	Rejected
C19	68000	770	330000	Salaried	Approved
C20	23000	555	125000	Unemployed	Rejected

Code:

```
import pandas as pd
import matplotlib.pyplot as plt

# Create the dataset
data = {
    "Customer_ID": ["C01", "C02", "C03", "C04", "C05", "C06", "C07", "C08", "C09", "C10",
```

```

        "C11","C12","C13","C14","C15","C16","C17","C18","C19","C20"],
    "Income": [45000,25000,60000,22000,50000,30000,70000,28000,55000,24000,
        65000,35000,48000,26000,75000,32000,52000,27000,68000,23000],
    "Credit_Score": [720,580,750,560,710,600,780,590,730,570,
        760,620,700,585,790,610,725,575,770,555],
    "Loan_Amount":
[200000,150000,300000,120000,250000,180000,350000,160000,270000,130000,
    320000,200000,240000,140000,400000,190000,260000,150000,330000,125000],
    "Employment_Type": ["Salaried","Self-employed","Salaried","Unemployed","Salaried",
        "Self-employed","Salaried","Self-employed","Salaried","Unemployed",
        "Salaried","Self-employed","Salaried","Unemployed","Salaried",
        "Self-employed","Salaried","Self-employed","Salaried","Unemployed"],
    "Loan_Status": ["Approved","Rejected","Approved","Rejected","Approved",
        "Rejected","Approved","Rejected","Approved","Rejected",
        "Approved","Rejected","Approved","Rejected","Approved",
        "Rejected","Approved","Rejected","Approved","Rejected"]
}

```

```
df = pd.DataFrame(data)
```

```
# Display dataset
```

```
print("Bank Loan Approval Dataset")
```

```
print(df)
```

```
# Dataset Information
```

```
print("\nDataset Information")
```

```
print(df.info())
```

```
# Statistical Summary
```

```
print("\nStatistical Summary")
```

```
print(df.describe())
```

```
# Loan Status Count
```

```
print("\nLoan Status Count")
```

```
print(df["Loan_Status"].value_counts())
```

```
# Average values by Loan Status
```

```
print("\nAverage Values")
```

```
print(df.groupby("Loan_Status")[["Income","Credit_Score","Loan_Amount"]].mean())
```

```
# Employment Type Count
```

```
print("\nEmployment Type Count")
```

```
print(df["Employment_Type"].value_counts())
```

```
# Income Bar Chart
```

```
plt.figure(figsize=(7,4))
```

```
plt.bar(df["Customer_ID"], df["Income"])
```

```
plt.title("Customer Income")
```

```
plt.xlabel("Customer ID")
```

```
plt.ylabel("Income")
```

```
plt.xticks(rotation=90)
plt.show()
```

Credit Score Bar Chart

```
plt.figure(figsize=(7,4))
plt.bar(df["Customer_ID"], df["Credit_Score"])
plt.title("Credit Score")
plt.xlabel("Customer ID")
plt.ylabel("Credit Score")
plt.xticks(rotation=90)
plt.show()
```

Loan Amount Bar Chart

```
plt.figure(figsize=(7,4))
plt.bar(df["Customer_ID"], df["Loan_Amount"])
plt.title("Loan Amount")
plt.xlabel("Customer ID")
plt.ylabel("Loan Amount")
plt.xticks(rotation=90)
plt.show()
```

Pie Chart - Loan Status

```
plt.figure(figsize=(5,5))
df["Loan_Status"].value_counts().plot(kind="pie", autopct="%1.1f%%")
plt.title("Loan Approval Status")
plt.ylabel("")
plt.show()
```

Conclusion

```
print("\nConclusion:")
print("Customers with higher income and higher credit scores are more likely to get loan approval.")
print("Most salaried customers are approved, while self-employed and unemployed customers are mostly rejected.")
```

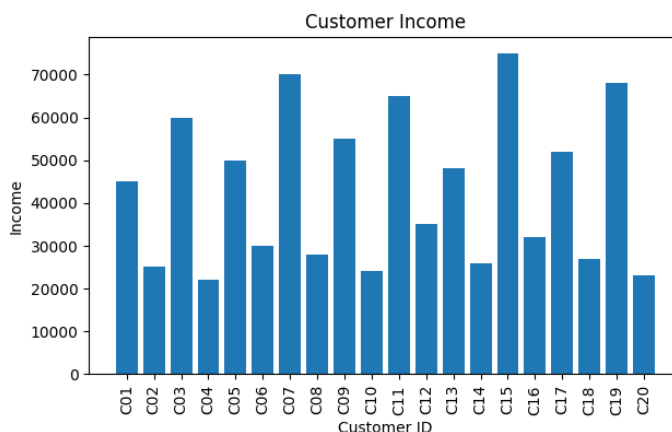
Output:

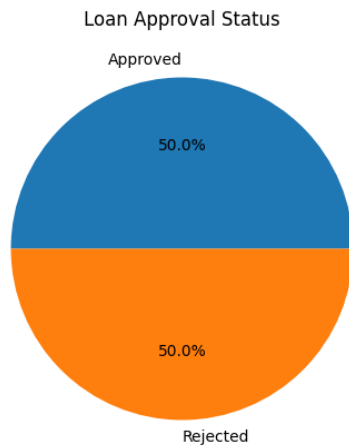
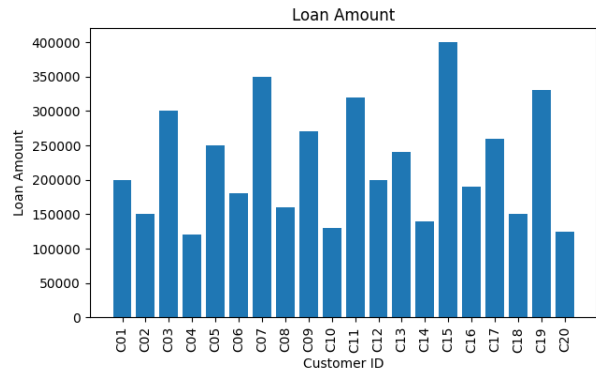
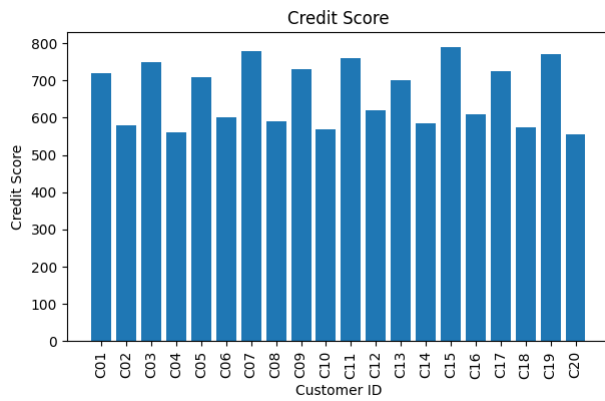
```
Saving healthcare.csv.xlsx to healthcare.csv (1).xlsx
First 5 Records
Patient_ID Age Sugar_Level BP BMI Disease_Status
0 P01 45 145 130 28 Yes
1 P02 32 98 118 23 No
2 P03 55 180 145 31 Yes
3 P04 28 90 110 22 No
4 P05 60 200 150 34 Yes

Shape: (20, 6)

Data Types:
Patient_ID object
Age int64
Sugar_Level int64
BP int64
BMI int64
Disease_Status object
dtype: object

Missing Values:
Patient_ID 0
Age 0
Sugar_Level 0
BP 0
...
Sugar_Level 40.334490
BP 14.523574
BMI 4.256265
dtype: float64
```





5. Agriculture Crop Yield Dataset

Question 5

An agricultural department wants to analyze crop yield based on rainfall, temperature, fertilizer usage, soil type, and crop yield. Perform EDA to identify the factors influencing crop production.

Farm_ID	Rainfall_mm	Temperature	Fertilizer_kg	Soil_Type	Crop_Yield_kg
F01	850	28	60	Loamy	3200
F02	600	32	45	Sandy	2100
F03	900	27	65	Loamy	3500
F04	500	34	40	Sandy	1800
F05	780	29	55	Clay	3000
F06	650	31	48	Sandy	2300
F07	920	26	70	Loamy	3700
F08	720	30	52	Clay	2800
F09	550	33	42	Sandy	1900
F10	800	28	58	Loamy	3100
F11	880	27	62	Loamy	3400
F12	620	32	46	Sandy	2200
F13	760	29	54	Clay	2950
F14	940	26	72	Loamy	3800
F15	580	33	44	Sandy	2000
F16	820	28	60	Clay	3150
F17	700	30	50	Clay	2700
F18	960	25	75	Loamy	3900
F19	540	34	41	Sandy	1850
F20	790	29	56	Clay	3050

Common EDA Tasks:

1. Identify the number of rows and columns.
2. Check data types of each column.
3. Find missing values and duplicate values.
4. Calculate mean, median, minimum, maximum, and standard deviation.
5. Draw suitable charts such as bar chart, histogram, scatter plot, and box plot.
6. Identify important patterns from the dataset.
7. Write 4 to 5 observations based on EDA.
8. Give a short conclusion from the analysis.

Code:

```
import pandas as pd
import matplotlib.pyplot as plt

# Create the dataset
data = {
    "Farm_ID": ["F01","F02","F03","F04","F05","F06","F07","F08","F09","F10",
               "F11","F12","F13","F14","F15","F16","F17","F18","F19","F20"],
    "Rainfall_mm": [850,600,900,500,780,650,920,720,550,800,
                    880,620,760,940,580,820,700,960,540,790],
    "Temperature": [28,32,27,34,29,31,26,30,33,28,
                   27,32,29,26,33,28,30,25,34,29],
    "Fertilizer_kg": [60,45,65,40,55,48,70,52,42,58,
                     62,46,54,72,44,60,50,75,41,56],
    "Soil_Type": ["Loamy","Sandy","Loamy","Sandy","Clay",
                  "Sandy","Loamy","Clay","Sandy","Loamy",
                  "Loamy","Sandy","Clay","Loamy","Sandy",
                  "Clay","Clay","Loamy","Sandy","Clay"],
    "Crop_Yield_kg": [3200,2100,3500,1800,3000,2300,3700,2800,1900,3100,
                     3400,2200,2950,3800,2000,3150,2700,3900,1850,3050]
}

df = pd.DataFrame(data)

# Display dataset
print("Agriculture Crop Yield Dataset")
print(df)

# Dataset Information
print("\nDataset Information")
print(df.info())

# Statistical Summary
print("\nStatistical Summary")
print(df.describe())

# Average values by Soil Type
print("\nAverage Values by Soil Type")
```

```
print(df.groupby("Soil_Type")[["Rainfall_mm", "Temperature", "Fertilizer_kg", "Crop_Yield_kg"]].mean())
```

```
# Highest Crop Yield
```

```
print("\nHighest Crop Yield")
```

```
print(df.loc[df["Crop_Yield_kg"].idxmax()])
```

```
# Rainfall Bar Chart
```

```
plt.figure(figsize=(7,4))
```

```
plt.bar(df["Farm_ID"], df["Rainfall_mm"])
```

```
plt.title("Rainfall")
```

```
plt.xlabel("Farm ID")
```

```
plt.ylabel("Rainfall (mm)")
```

```
plt.xticks(rotation=90)
```

```
plt.show()
```

```
# Fertilizer Bar Chart
```

```
plt.figure(figsize=(7,4))
```

```
plt.bar(df["Farm_ID"], df["Fertilizer_kg"])
```

```
plt.title("Fertilizer Usage")
```

```
plt.xlabel("Farm ID")
```

```
plt.ylabel("Fertilizer (kg)")
```

```
plt.xticks(rotation=90)
```

```
plt.show()
```

```
# Crop Yield Bar Chart
```

```
plt.figure(figsize=(7,4))
```

```
plt.bar(df["Farm_ID"], df["Crop_Yield_kg"])
```

```
plt.title("Crop Yield")
```

```
plt.xlabel("Farm ID")
```

```
plt.ylabel("Crop Yield (kg)")
```

```
plt.xticks(rotation=90)
```

```
plt.show()
```

```
# Pie Chart - Soil Type
```

```
plt.figure(figsize=(6,6))
```

```
df["Soil_Type"].value_counts().plot(kind="pie", autopct="%1.1f%%")
```

```
plt.title("Soil Type Distribution")
```

```
plt.ylabel("")
```

```
plt.show()
```

```
# Conclusion
```

```
print("\nConclusion:")
```

```
print("Higher rainfall and fertilizer usage increase crop yield.")
```

```
print("Loamy soil produces the highest crop yield.")
```

```
print("Lower temperatures are associated with better crop production.")
```


Output:

Dataset Information

```
<class 'pandas.core.frame.DataFrame'> RangeIndex: 20 entries, 0 to 19 Data columns (total 6 columns):  #   Column      Non-Null Count  Dtype ---  --  0   Farm_ID     20 non-null     object  1   Rainfall_mm 20 non-null     int64  2   Temperature 20 non-null     int64  3   Fertilizer_kg 20 non-null    int64  4   Soil_Type   20 non-null     object  5   Crop_Yield_kg 20 non-null    int64 dtypes: int64(4), object(2) memory usage: 1.1+ KB None
```

Statistical Summary

	Rainfall_mm	Temperature	Fertilizer_kg	Crop_Yield_kg
count	20.000000	20.000000	20.000000	20.000000
mean	741.800000	25.500000	54.750000	2820.000000
std	144.135136	2.762855	19.597516	681.487213
min	500.000000	25.000000	40.000000	1800.000000
25%	615.000000	27.750000	45.750000	2175.000000
50%	770.000000	29.000000	54.000000	2975.000000
75%	857.000000	32.000000	60.000000	3250.000000
max	960.000000	34.000000	75.000000	3900.000000

Average Values by Soil Type

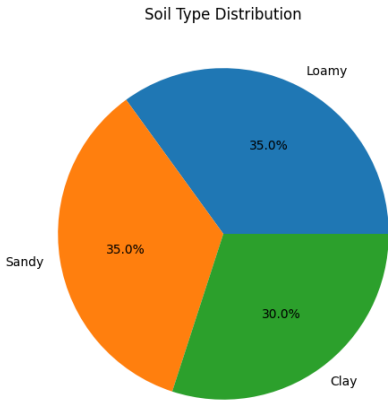
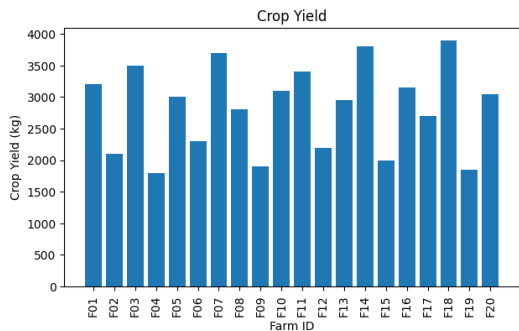
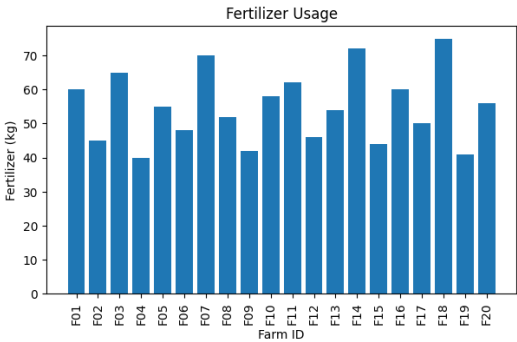
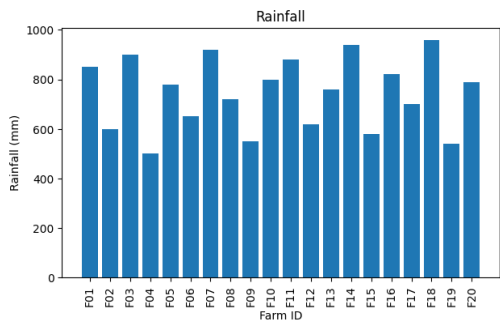
Soil_Type	Rainfall_mm	Temperature	Fertilizer_kg	Crop_Yield_kg
Clay	761.666667	29.333333	54.000000	2943.666667
Loamy	802.857143	26.714286	66.000000	3514.285714
Sandy	577.142857	32.714286	43.714286	2021.428571

Highest Crop Yield

Farm_ID	F18
Rainfall_mm	960
Temperature	25
Fertilizer_kg	75
Soil_Type	Loamy
Crop_Yield_kg	3900

Agriculture Crop Yield Dataset

	Farm_ID	Rainfall_mm	Temperature	Fertilizer_kg	Soil_Type	Crop_Yield_kg
0	F01	850	28	60	Loamy	3200
1	F02	600	32	45	Sandy	2100
2	F03	900	27	65	Loamy	3500
3	F04	500	34	40	Sandy	1800
4	F05	780	29	55	Clay	3000
5	F06	650	31	48	Sandy	2300
6	F07	920	26	70	Loamy	3700
7	F08	720	30	52	Clay	2800
8	F09	550	33	42	Sandy	1900
9	F10	800	28	58	Loamy	3100
10	F11	880	27	62	Loamy	3400
11	F12	620	32	46	Sandy	2200
12	F13	760	29	54	Clay	2950
13	F14	940	26	72	Loamy	3800
14	F15	580	33	44	Sandy	2000
15	F16	820	28	60	Clay	3150
16	F17	700	30	50	Clay	2700
17	F18	960	25	75	Loamy	3900
18	F19	540	34	41	Sandy	1850
19	F20	790	29	56	Clay	3050



RUBRICS FOR CALCULATION

Exploratory Data Analysis-Based Practical Assessment					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Dataset Understanding	20	Clearly explains dataset source, features, target variables, data types, and problem context with strong clarity	Explains dataset, features, and variables with adequate clarity	Basic explanation of dataset with limited understanding of variables	Poor understanding of dataset; variables and problem context are unclear
Data Cleaning	25	Effectively identifies and handles missing values, duplicates, incorrect entries, and outliers using suitable methods	Handles most missing values, duplicates, and errors correctly	Performs basic cleaning but misses some important data quality issues	Minimal or incorrect data cleaning; major errors remain in the dataset
Data Analysis and Statistical Summary	20	Provides detailed descriptive statistics, comparisons, patterns, and meaningful interpretations	Provides relevant summary statistics and basic interpretation	Provides limited statistics with weak explanation	Lacks statistical analysis or gives incorrect interpretation
Data Visualization	20	Uses highly appropriate charts with proper labels, titles, and clear visual interpretation	Uses suitable charts with mostly clear labels and explanation	Uses limited or basic charts with unclear interpretation	Poor or incorrect visualizations; charts do not support analysis
Insight Generation and Reporting	15	Presents strong insights, conclusions, and recommendations based on data evidence	Presents clear findings and conclusions with reasonable support	Provides basic findings but lacks depth or proper justification	Findings are unclear, unsupported, or not related to the data

CO1-ASSESSMENT TOOL -2 Concept Map**QUESTIONS : 5****MARKS : 50**

Q No	Dataset Understanding (20%)	Data Cleaning (25%)	Data Analysis and Statistical Summary (20%)	Data Visualization (20%)	Insight Generation and Reporting (15%)	Total Score (100%)
1	/2.5	/2.5	/2.0	/2.0	/1.0	/10
2	/2.5	/2.5	/2.0	/2.0	/1.0	/10
3	/2.5	/2.5	/2.0	/2.0	/1.0	/10
4	/2.5	/2.5	/2.0	/2.0	/1.0	/10
5	/2.5	/2.5	/2.0	/2.0	/1.0	/10
Overall Rubrics Value	/12.5	/12.5	/10	/10	/10	/50
	/25	/25	/20	/20	/10	/100